

Comunicação por Eventos e Mensageria Assíncrona

Sistemas Distribuídos - Prof. Marcos Rogério Zibordi - Turma 57-06-B

Integrantes: Fernando Petri (RA 20240770)

Desafio inicial

Com mensageria assíncrona o "Finalizar Compra" fica quase instantâneo porque não precisa fazer tudo dentro da mesma operação. No modelo síncrono o cliente espera pagamento, estoque, nota fiscal e e-mail terminarem, uns 5 segundos. No assíncrono o checkout só grava o pedido e publica o evento "PedidoCriado", o que leva milissegundos. Depois pagamento, estoque, nota e e-mail tratam esse evento por baixo dos panos, sem o cliente esperar.

Desafio 1

1.1

- a) PedidoCriado - Evento
- b) EnviarEmail - Comando
- c) PagamentoAprovado - Evento
- d) GerarPDF - Comando

1.2 O verbo no passado indica algo que já aconteceu (um fato). É o usado em eventos, onde o sistema avisa que aconteceu algo e quem quiser pode reagir. O verbo no imperativo é usado em comandos, que são uma ordem para alguém executar uma ação, e essa ordem pode falhar ou ser recusada.

1.3

```
{  
  "id": "0f09efff-e8ae-442e-be55-906456c219c5",  
  "timestamp": "2026-09-20T17:48:19Z",  
  "cliente": "José da Silva Sanches",  
  "valor": 1240.23  
}
```

Desafio 2

No Pub/Sub o sistema publica o evento "PedidoSaiuParaEntrega" uma vez só no broker, e o broker entrega uma cópia para cada consumidor (push, GPS, SMS e dashboard de BI) ao mesmo tempo. Quem publica não conhece nem chama nenhum dos consumidores, só avisa o broker que aconteceu. Por isso os serviços ficam desacoplados: dá para adicionar outro consumidor sem mexer no produtor, e se um deles cair os outros continuam funcionando.

Sobre o Netflix e o Google: o Netflix é uma empresa independente e não é do Google (o Google tem o YouTube). O Pub/Sub também é só um padrão de arquitetura, o Google Cloud Pub/Sub é apenas um produto que usa esse nome.

Desafio 3

Broker	Protocolo / padrão	Melhor caso de uso
RabbitMQ	AMQP, filas e exchanges	Filas de tarefas assíncronas e mensageria empresarial, com roteamento, DLQ, pub/sub e fanout.
Apache Kafka	Event streaming / log distribuído	Volume muito alto de dados, event sourcing, replay de eventos, sistemas financeiros e análise em tempo real.
AWS SQS / SNS	Gerenciado na nuvem (HTTP/REST)	Microserviços e serverless na AWS. SQS é a fila e SNS é o pub/sub.

Desafio 4

- Poison message: se não tiver tratamento de erro o consumidor tenta processar a mensagem toda hora e falha, criando um loop infinito, e ela ainda pode travar as mensagens que vêm depois.
- Retry exponencial: dá tempo do serviço com problema se recuperar. Tentar direto toda hora só empilha requisição e piora a situação.
- DLQ: depois de esgotar as tentativas a mensagem vai para a DLQ. Assim ela não trava a fila principal e dá para analisar depois o que deu errado.

Desafio 5

Circuit Breaker: é um padrão de resiliência. Se um serviço está falhando, o circuito abre e as chamadas param de ir até ele por um tempo, evitando que a falha se espalhe para os outros serviços em sistemas distribuídos.

Outbox Pattern: o problema é gravar o débito no banco e publicar o evento no broker, que são dois sistemas diferentes e um pode falhar sem o outro. No Outbox a mensagem é salva no banco na mesma transação do débito, então ou salva os dois ou nenhum. Depois um processo lê essa tabela e publica no broker. Assim a mensagem não se perde e o sistema chega na consistência eventual.